

Week 7 Workshop

9-11 minutes

CMP9134 — Week 7 Workshop: Containerisation

20 March 2026 · Prof. Marc Hanheide

You will spend this session getting a real multi-container application running, reading it to understand how the pieces fit together, and then extending it. By the end you will have implemented new robot control endpoints and watched your CI pipeline test them automatically.

Before you start

1. Use the template repository

Go to:

https://github.com/UoL-SoCS/CMP9134_2526_template

Click **Use this template** (top-right) → **Create a new repository** → choose yourself as owner, and name your repository → choose visibility **private** (advised to avoid any possible collusion risk of your work, but up to you) → click **Create Repository**

This gives you your own repository under your GitHub account, using the template.

All your work today (and possibly your assessment work?) should live in your own repository. This is *your* playground, start playing...

2. Open in VS Code Dev Container

Open VSCode and clone **your repository** to your machine and open it.

VS Code will detect the `.devcontainer/` folder and show a notification:

("Reopen in Container")

Click it. VS Code will:

- Build the Docker images for the backend and frontend
- Pull the pre-built robot simulator image

- Start all three containers
- Install all recommended extensions inside the container

This takes **2-4 minutes** the first time. Grab a coffee.

If the notification doesn't appear: open the Command Palette (Ctrl+Shift+P / Cmd+Shift+P) and run: Dev Containers: Reopen in Container

3. Find your forwarded ports

Once the dev container is running, click the **Ports** tab in the VS Code bottom panel (next to Terminal):

Label	Port	What it is
Frontend	80	The web dashboard — your main UI
Backend API (uvicorn)	8000	The FastAPI backend you will extend
Robot API (provided)	5000	The virtual robot simulator

VS Code adds a  icon next to each forwarded port. Click it to open that service in your browser.

On GitHub Codespaces? You can also try GitHub Codespaces, which is basically VSCode in the cloud. A devcontainer allows you to define the working environment so a cloud "computer" is set up exactly with your development environments. The URLs will be something like `https://xxx-8000.app.github.dev` instead of `localhost`. Use whatever URL VS Code gives you — the port numbers are the same.

Task 0 — Verify everything is running

Open a **terminal OUTSIDE of the dev container** (Terminal → New Terminal) and run:

```
docker ps
```

(or simply check Docker Desktop on Windows)

You should see three services listed. Note their state — are they all running/healthy?

Now, VSCode, open each of these in your browser (use the Ports tab):

Port	Forwarded Address
○ Frontend (frontend:80)	localhost:57780
○ Robot API (provided) (robot-api:50...	localhost:5000
○ Backend API (uvicorn) (8000)	localhost:8000

URL	What you should see
Frontend	The basic dashboard — position, battery, status cards
Backend API	go to /docs, FastAPI interactive API docs (Swagger UI)
Robot API	go to /docs, Robot simulator API docs

Checkpoint questions — answer these before moving on:

1. How many Docker containers are running right now? Run `docker compose ps` to check.
 2. Open the dashboard. Is the robot status showing as connected?
 3. Open the robot API docs. What endpoints does the robot expose? Does it meet your expectations.
-

Task 1 — Explore and Read the stack before you touch it

The template is deliberately annotated — every key line in `docker-compose.yml`, `devcontainer.json`, and the `Dockerfile` has a comment explaining what it does and why.

Open these files in VS Code and read through them:

- `docker-compose.yml`
- `.devcontainer/devcontainer.json`
- `.devcontainer/docker-compose.devcontainer.yml`
- `backend/Dockerfile`

As you read, answer the following questions. Write your answers in a scratch file or a comment — they will be useful when writing your assessment report. You have to use a search engine to do some research yourself!

Questions on `docker-compose.yml`:

1. There are three services: `robot-api`, `backend`, and `frontend`. How does the backend know the URL of the robot API? (*Hint: look at the `environment` section of the backend service.*)
 2. The backend uses `depends_on` with `condition: service_started`. The frontend uses `condition: service_healthy`. What is the difference, and why does the frontend need the stricter condition?
 3. The robot API exposes port 5000 — but there is no `ports` mapping for it in `docker-compose.yml`. How can the backend still reach it?
 4. There is a commented-out `database` service. What image would it use? What would you need to uncomment to make it persist data between restarts?
-

Questions on the `Dockerfile` (`backend/Dockerfile`):

5. The `Dockerfile` has three stages: `base`, `development`, and `production`. Which stage does the devcontainer use? Which does the CI pipeline use? (*Hint: check `docker-compose.devcontainer.yml`, `docker-compose.yml`, and `.github/workflows/ci.yml`.*)
 6. The `production` stage creates a non-root user called `gcs` (see `backend/Dockerfile`). Why is this good practice?
 7. The `development` stage defines the command to execute upon start with `CMD ["uvicorn", "main:app", ... "--reload"]`. But in the dev container, `uvicorn` is not running by default. What command overrides this, and why? (*Hint: `docker-compose.devcontainer.yml`.*)
-

Task 2 — Extend the robot client

Open `backend/robot_client.py`.

The `get_status()` method is already implemented. The `move()` and `reset()` methods exist but raise `NotImplementedError`:

```
async def move(self, x: int, y: int) -> dict[str, Any]:
    raise NotImplementedError("Move command not implemented yet")

async def reset(self) -> dict[str, Any]:
    raise NotImplementedError("Reset command not implemented yet")
```

Your job is to implement at least *one* of them.

Step 2a — Check the robot API docs

Open `robot-api:5000/docs` (from "ports" in VSCode, this is *likely* <http://localhost:5000/docs>) in your browser. Find the `POST /api/move` endpoint. It should look like this:

Send a navigation command to the robot.

Moves the robot to the specified (x, y) coordinates.

Validation Rules:

- Coordinates must be within the 0-20 grid range.
- Robot cannot move if the battery is empty (0%).
- Robot will stop if it encounters an obstacle.

Click it → **Try it out** → fill in x and y values → **Execute**.

Note:

- What is the request body format? (JSON with x and y fields)
- What does a successful response look like?
- What happens if you give coordinates outside the 0-20 grid?

Do the same for `POST /api/reset`.

Step 2b — Implement `move()` or `reset()`

In `robot_client.py`, replace the `NotImplementedError` with a real implementation.

Model it on `get_status()` — it uses `httpx.AsyncClient` to send a GET request. `move()` should send a **POST** request to `/api/move` with a JSON body:

```
{"x": x, "y": y}
```

Make sure to:

- Raise `RobotConnectionError` on any exception (match the pattern in `get_status()`)
 - Return the parsed JSON response
-

Step 2c — Add endpoints to main.py

Open backend/main.py. You can see the stub route skeletons in the comments at the bottom.

Add a new route for your new function (either move or reset):

e.g. start with

```
@app.post("/api/move")
async def move_robot(x: int, y: int):
    """Send a move command to the robot."""
    ...
```

Hot reload: Because the dev container runs uvicorn with `--reload`, your changes take effect as soon as you save the file. No restart needed.

(You will need to start uvicorn first — see below if you haven't already.)

Step 2d — Start the backend

The dev container keeps the container alive but does **not** start uvicorn automatically (so you can make changes before it starts). In your terminal:

```
cd /workspace/backend
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Leave this running in the terminal. You should see:

```
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      Started reloader process
```

Step 2e — Test your new endpoints

Open backend API port, likely at <http://localhost:8000/docs> in your browser. You should now see:

- GET /api/status
- POST /api/move, or
- POST /api/reset

Test each one using the **Try it out** button in Swagger.

For /api/move, try:

- Valid coordinates: x=5, y=3
 - Out-of-range coordinates: x=25, y=0
-

Stretch: Handle the chaos monkey

The robot has a **chaos monkey** — it randomly fails communication. Your `move()` or `reset()` implementations will

sometimes fail.

Explore how to add a basic retry logic to `robot_client.py`.

Why does this matter? Your assessment brief explicitly requires robust error handling for the chaos monkey. The *Critical Reflection* section (S1, 20% of marks) expects you to discuss engineering decisions like this.

Task 3 — Commit and watch CI run

You have extended the backend. Now push your changes and watch the CI pipeline test them.

Step 3a — Commit your changes

In your terminal (you can open a second terminal tab):

```
cd /workspace
git add backend/robot_client.py backend/main.py
git commit -m "implement move and reset robot endpoints"
git push
```

Or you simply use the built-in VSCode tools ;-)

Step 3b — Watch the Actions run

Go to your forked repository on GitHub → **Actions** tab.

You will see a workflow called **CI — Build, Test & Push** running. It has two jobs:

Click into each job and read through the steps.

Questions:

1. The test job runs on **two Python versions** simultaneously. Why?
2. The compose integration test hits `http://localhost:8080/health`. Which service handles that request? Trace the path.
3. Do your new `/api/move` and `/api/reset` endpoints get tested by the CI pipeline? If not, what kind of test would you write to cover them?

Ports (inside the dev container):

Service	Port	URL
Frontend dashboard	80	<code>http://localhost:80</code>
Your FastAPI backend	8000	<code>http://localhost:8000/docs</code>
Robot simulator API	5000	<code>http://localhost:5000/docs</code>